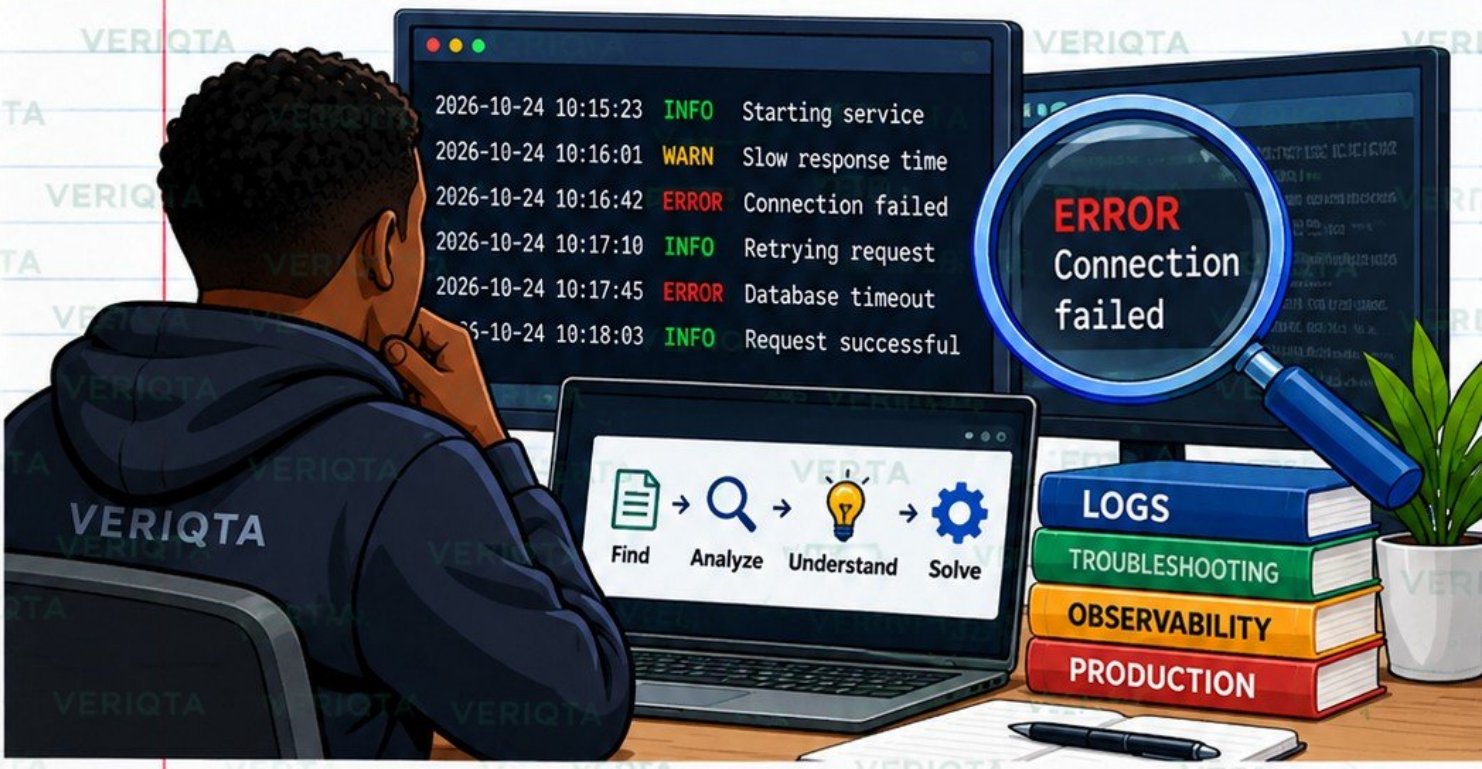
VERIQTA

Reading Logs Like a Detective:

A Beginner's Playbook



READ LOGS EFFECTIVELY

FIND ROOT CAUSE FAST

TROUBLESHOOT REAL ISSUES

BUILD PRODUCTION CONFIDENCE

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

Reading Logs Like a Detective

1. What Are Logs?

- Logs are **timestamped records** of events
- that happen in a system, application or infrastructure.
- They show **what** the system is doing, **when** it happens, and the **result**.
- Logs help engineers understand **normal behaviour**, find **problems** and investigate **failures**.

2. Why Applications Produce Logs

- To record important events
- To track user activity
- To show errors and warnings
- To help with debugging and troubleshooting
- To monitor system health
- To provide an audit trail for security
- To understand performance issues

3. How Engineers Use Logs

Collect Logs → **Search Filter** → **Analyze Patterns** → **Find Root Cause** → **Take Action**

- **Collect Logs**: From application, server, or cloud services
- **Search Filter**: Find specific events, errors or time periods
- **Analyze Patterns**: Look for trends, repeated errors or unusual activity
- **Find Root Cause**: Identify what happened and why
- **Take Action**: Fix the issue and verify the system

4. Understanding a Log Entry

```
2026-10-24 14:03:12 ERROR [auth-service] UserService - Failed to authenticate user john@example.com - Invalid password
```

- **Timestamp**: When the event happened
- **Log level**: INFO, WARN, ERROR, etc.
- **Service/Component**: Which part of the system generated the log
- **Source**: Class or module that created the log
- **Message**: Details about what happened

5. Common Log Levels

DEBUG	Detailed information for troubleshooting
INFO	Normal operations information
WARN	Something unusual happened
ERROR	A problem occurred that needs attention
FATAL	A serious error, system may stop

6. Real World Example

```
2026-10-24 14:00:01 INFO [api] Server started on port 8080
2026-10-24 14:02:15 INFO [api] Received login request for user john@example.com
2026-10-24 14:02:16 WARN [api] Invalid password attempt for user john@example.com
2026-10-24 14:03:12 ERROR [api] Account locked after 5 failed attempts
```

7. Practice: Read These Logs and Answer

1. 2026-10-24 10:15:20 INFO [web] Server started
2. 2026-10-24 10:16:05 INFO [db] Connected to database
3. 2026-10-24 10:17:12 ERROR [db] Connection lost
4. 2026-10-24 10:17:30 INFO [web] Retrying database connection
5. 2026-10-24 10:18:05 INFO [db] Connected to database

Questions:

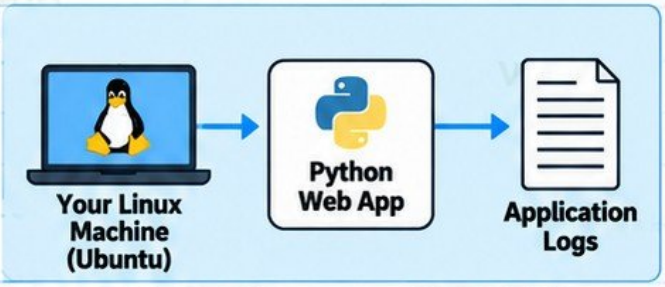
1. What happened first?
2. When did the database connection fail?
3. What action did the system take after the failure?
4. When was the database connection restored?
5. What does this tell you about the system behaviour?

Key Takeaway: Logs tell a story. Read them in order, look for errors, and use the timestamps to understand what happened and why.

Build Your First Log Investigation Lab

1. Set Up a Safe Linux Practice Environment

- Use a local Linux machine or a virtual machine with Ubuntu.
- This lab is safe and does not affect any production system.
- You will install a small web application that writes logs so you can practice investigating real log entries.



2. Update Your System

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3 python3-pip \
   curl git
```

What This Does

- Updates your package list.
- Upgrades existing packages.
- Installs Python 3, pip, curl and git.
- These tools are needed to run our sample application.

3. Create a Project Directory

```
mkdir -p ~/veriqta-log-lab
cd ~/veriqta-log-lab
pwd
  /home/ubuntu/veriqta-log-lab
```

Why This Matters

- Keeps your lab organized.
- All files, logs and scripts will be inside this directory.
- You can easily delete it later if needed.

4. Create a Simple Python Web Application

```
cat > app.py << 'EOF'
from flask import Flask, request
import logging
from datetime import datetime

app = Flask(__name__)

logging.basicConfig(
   filename='app.log',
   level-logging.INFO,
   format='%(asctime)s %(levelname)s %(message)s'
)

@app.route('/')
def home():
    client_ip = request.remote_addr
    user_agent = request.headers.get('User-Agent')
    logging.info(f"Request from {client_ip} - {user_agent}")
)   return "VERIQTA Log Lab is running!\n"

if __name__ == '__main__':
     app.run(host='0.0.0.0', port=5000)
EOF
```

What This Application Does

- Runs a simple web server using Flask.
- Logs each request to a file called app.log.
- Records the client IP and user agent.
- Helps you generate real log entries for investigation.

5. Install Flask and Run the Application

```
pip3 install flask
python3 app.py
```

6. Generate Requests and Check Logs

```
curl http://localhost:5000
curl http://localhost:5000
curl http://localhost:5000

tail -n 5 app.log
```

Expected Output in app.log

```
2026-10-24 12:15:01,123 INFO Request from 127.0.0.1 - curl/8.5.0
2026-10-24 12:15:05,456 INFO Request from 127.0.0.1 - curl/8.5.0
2026-10-24 12:15:09,789 INFO Request from 127.0.0.1 - curl/8.5.0
```

Practice This

- Run the application and generate at least 5 requests.
- Open app.log and confirm the entries appear.
- Modify the message to include the URL path.
- Try accessing http://localhost:5000/test and check the new log entry.

Page 03: Understand Every Part of a Log Entry

Learn timestamps, severity levels, service names, process identifiers, request IDs, and error messages.

Practice: Annotate real output and distinguish symptoms from possible causes.

1. Parts of a Log Entry



2. Common Log Formats

Plain Text Log

```
2026-10-23 14:32:10 [INFO] app[9876]:
req_id=8f3c2d4a User login successful for user=john
```

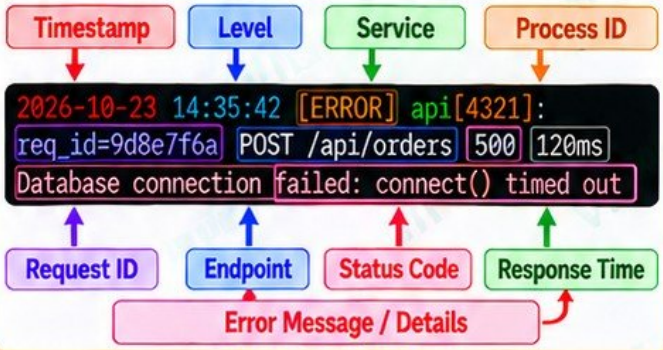
JSON Log (Structured Log)

```
{
  "timestamp": "2026-10-23T14:32:10Z",
  "level": "ERROR",
  "service": "api",
  "pid": 9876,
  "request_id": "8f3c2d4a",
  "message": "Database connection failed",
  "error": "connect() failed (111: Connection refused)"
}
```

3. Common Severity Levels

Level	Meaning
TRACE	Very detailed information (debugging).
DEBUG	Detailed information for troubleshooting.
INFO	Normal operational messages.
WARN	Something unexpected happened, but the service is still working.
ERROR	A failure occurred and an operation did not complete.
FATAL	A critical error that may stop the service.

4. Annotated Real Log Entry



5. Identify Key Information

```
2026-10-23 14:40:01 [WARN] worker[5678]:
req_id=4a1b2c3d Retrying request to /api/payments
(attempt 2 of 3)
```

- **Timestamp:** 2026-10-23 14:40:01
- **Severity:** WARN (a problem, but service still running)
- **Service:** worker
- **Process ID:** 5678
- **Request ID:** 4a1b2c3d
- **Message:** Retrying request to /api/payments
- **Additional detail:** attempt 2 of 3

6. Symptoms vs Possible Causes

Symptom (from logs)	Possible Causes
Connection refused	Backend service is down Wrong port or host Firewall or network issue
Timed out	Service is slow or overloaded Network latency Backend not responding
500 Internal Server Error	Application error Database error Bug in application code
404 Not Found	Incorrect endpoint Missing route Wrong version
Permission denied	File permission issue Missing credentials Wrong user or role

7. Practice (Hands-On)

1. Read the sample log entries.
2. Annotate each part (timestamp, level, service, PID, request ID, message).
3. Identify the main symptom.
4. Suggest at least one possible cause.
5. Repeat with different log formats (plain text and JSON).

8. Key Takeaways

- Every part of a log entry provides useful information.
- Severity levels help you quickly identify serious issues.
- Request IDs let you trace a single request across multiple services.
- The message tells you what happened, but you must investigate the possible cause.
- Always distinguish the symptom from the real root cause.

Page 04: Find Logs on a Linux Server

Explore /var/log, application log files, permissions, log rotation, and the difference between file-based logs and systemd journals.

1. What Are Linux Logs?

- Logs are records of system events, application activity and errors.
- They help you troubleshoot problems, understand what happened and monitor system health.
- Logs can be stored in files or managed by systemd journals.

2. Main Log Locations

/var/log

- apache2/ (Web server logs)
- nginx/ (Nginx logs)
- mysql/ (MySQL logs)
- auth.log (Authentication logs)
- syslog (General system logs)
- kern.log (Kernel logs)
- messages (System messages)
- secure (Security-related logs)
- dmesg (Boot and hardware logs)
- journal/ (systemd journal files)

3. List Available Log Files

```
# View contents of /var/log
ls -l /var/log
# List only directories
ls -ld /var/log/*/
# Find log files (top 20)
sudo find /var/log -type f | head -20
```

4. View Log Content

```
# View last 10 lines of a log file
sudo tail -n 10 /var/log/syslog
# Follow a log file in real time
sudo tail -f /var/log/syslog
# View first 10 lines
sudo head -n 10 /var/log/auth.log
```

5. Check Permissions

```
# Check file permissions
ls -l /var/log/auth.log

# Check who owns the file
ls -l /var/log/nginx/
```

Many logs are owned by root and require sudo to view because they contain sensitive information.

6. Log Rotation

- Log files grow over time. logrotate automatically compresses, rotates and removes old logs.
- Configuration files are usually in /etc/logrotate.conf and /etc/logrotate.d/
- Rotation prevents disk space issues.
- You do not need to manually delete logs.

7. File-Based Logs vs systemd Journals

File-Based Logs

- Stored as text files in /var/log
- Can be viewed with cat, tail, less, grep
- Common for applications (Nginx, Apache, MySQL)
- Managed by logrotate
- Persist until rotated or deleted

systemd Journals

- Managed by systemd (journald)
- View with journalctl
- Stores structured logs with metadata
- Includes logs from services, kernel, and boot
- Usually stored in /var/log/journal/

8. Practice: Locate Logs (Read-Only)

1. List all log directories in /var/log.
2. Find the last 10 lines of /var/log/syslog.
3. Check the size of the largest log file in /var/log.
4. View the most recent systemd logs.
5. Identify available application logs (e.g. nginx, apache, mysql).
6. **Do NOT modify or delete any files.**

Page 05: Read Logs with cat, less, head, and tail

Learn how to inspect large log files efficiently without flooding the terminal.
Practice: Use `tail -n 20 app.log` and `less app.log` to locate a simulated failure.

1. Why Log Files Can Be Large

- Applications, services, and the OS write logs continuously.
- Logs help you find errors, troubleshoot issues, and understand what happened.
- Large log files can be thousands or millions of lines, so you should not open them with `cat` every time.

2. Common Log Files

Log File	What It Contains
/var/log/syslog	General system logs
/var/log/auth.log	Authentication logs
/var/log/nginx/	Nginx access and error logs
/var/log/apache2/	Apache access and error logs
/var/log/mysql/	MySQL logs
/var/log/	Application and service logs

3. Read the Entire File with cat

```
user@server:~$ cat /var/log/syslog
Oct 23 10:12:01 server systemd[1]: Started app.service
Oct 23 10:12:02 server app[2345]: Application started
Oct 23 10:12:15 server app[2345]: Processing request
```

- Use `cat` only for small log files.
- It prints the entire file and can flood your terminal for large logs.

4. View the Last Lines with tail

```
user@server:~$ tail -n 20 /var/log/app.log
Oct 23 14:55:10 server app[2345]: INFO User login
Oct 23 14:55:22 server app[2345]: WARN Slow query
Oct 23 14:55:30 server app[2345]: ERROR Database
                                  connection failed
```

- `tail -n 20` shows the last 20 lines.
- Useful for quickly checking recent errors.
- Commonly used for live monitoring.

5. View the First Lines with head

```
user@server:~$ head -n 20 /var/log/app.log
Oct 23 10:00:01 server app[2345]: Starting app
Oct 23 10:00:02 server app[2345]: Loading config
Oct 23 10:00:03 server app[2345]: Connected to DB
```

- `head -n 20` shows the first 20 lines.
- Useful for checking how a log file starts.
- Helps you understand the context.

6. Read Large Logs with less

```
user@server:~$ less /var/log/app.log
```

- Use `less` to read large files page by page.
- Scroll using arrow keys.
- Press `Space` to go down a page.
- Press `b` to go back a page.
- Press `q` to exit.
- You can also search by typing `/error` and press `Enter`.

7. Practical Example: Find a Failure

```
user@server:~$ tail -n 20 app.log
Oct 23 15:10:12 server app[2345]: INFO Processing order
Oct 23 15:10:15 server app[2345]: ERROR Payment failed
Oct 23 15:10:15 server app[2345]: ERROR Invalid card
Oct 23 15:10:16 server app[2345]: INFO Retrying...
```

1. Use `tail -n 20 app.log` to check recent errors.
2. Use `less app.log` to explore more context around the error.
3. Look for `ERROR` or `WARN` messages.
4. Identify the timestamp and service that reported the error.

8. Practice (Read-Only)

1. List the size of /var/log/syslog.
2. Use `head -n 20 /var/log/syslog`.
3. Use `tail -n 20 /var/log/syslog`.
4. Open /var/log/syslog with `less` and search for the word `error`.
5. Find an application log (nginx, apache, mysql) on your server and view the last 20 lines.
6. Locate a simulated failure in app.log using `tail -n 20` and `less`.
7. Do NOT modify or delete any log files.

Page 06: Watch Logs While an Application Runs

Understand live log streaming and why the timing of events matters.

Practice: Run `tail -f app.log` in one terminal while generating application activity in another.

1. What Is Live Log Streaming?

- Live log streaming shows new log entries as they are written to a file.
- It helps you see what is happening in real time.
- Useful for debugging applications, services and system events.
- The timing of events matters because logs show the exact order of what happened.

2. How tail -f Works

- `tail -f` continuously reads the end of a file.
- It waits for new lines and displays them immediately.
- It does not reload the entire file.
- Useful for live monitoring during deployments, troubleshooting, and testing.

3. Basic Live Log Monitoring

```
user@server:~$ tail -f /var/log/app.log
Oct 23 15:10:01 INFO  Starting application
Oct 23 15:10:05 INFO  Connected to database
Oct 23 15:10:08 INFO  User login successful
Oct 23 15:10:12 ERROR Payment failed
Oct 23 15:10:15 INFO  Retrying request
```

- Press `Ctrl + C` to stop.
- You will only see new log entries.

4. Generate Application Activity

```
user@server:~$ echo "Test request" >> app.log
user@server:~$ echo "Processing..." >> app.log
user@server:~$ echo "ERROR Something failed" \
                    >> app.log
```

- Run commands like this in another terminal.
- New entries will appear immediately in the terminal running `tail -f`.
- This helps you confirm that the application is writing logs correctly.

5. Common Options

Command	What It Does	When to Use It
`tail -f app.log`	Show new log entries live	Monitor logs in real time
`tail -n 50 app.log`	Show last 50 lines and exit	Quickly check recent logs
`tail -F app.log`	Follow even if file is rotated	Use with logrotate
`less app.log`	View logs page by page	Read large files manually

6. Real-World Example



7. Why Timing of Events Matters

- Logs show the exact sequence of events.
- They help you understand what happened before a failure.
- Useful for debugging errors, performance issues, and service restarts.
- Helps you correlate application events with system logs and metrics.

8. Practice (Hands-On)

1. Open one terminal and run: `tail -f app.log`
2. Open a second terminal and generate log entries using `echo` or run a simple script.
3. Observe how new entries appear in real time.
4. Identify an `ERROR` entry and note the timestamp.
5. Stop the live stream with `Ctrl + C`.

Page 07: Search Logs with grep

Find specific error messages, request IDs, and repeated failures.
Learn case-insensitive searches, line numbers, and context lines.

Practice: Use `grep -n -C 2 'ERROR' app.log` to inspect what happened around a failure.

1. What is grep?

- grep searches for a specific pattern (text) in files and shows matching lines.
- It is very useful for finding errors, request IDs, and important events in log files.
- You can search by exact words, ignore case, show line numbers, and display context lines.

2. Basic Syntax

```
grep [options] "pattern" file.log
```

- **options:** control how the search works
- **pattern:** the text you are looking for
- **file.log:** the log file to search

3. Common grep Options

Option	What It Does	Example
-n	Show line numbers	grep -n "ERROR" app.log
-i	Case-insensitive search	grep -i "error" app.log
-v	Show lines that do NOT match	grep -v "DEBUG" app.log
-C N	Show N lines of context (before and after)	grep -C 2 "ERROR" app.log
-A N	Show N lines after (match)	grep -A 2 "ERROR" app.log
-B N	Show N lines before (match)	grep -B 2 "ERROR" app.log
-r	Search recursively (subdirectories)	grep -r "ERROR" /var/log

4. Practical Examples

Find all error lines

```
grep "ERROR" app.log
```

Case-insensitive search

```
grep -i "error" app.log
```

Show line numbers

```
grep -n "ERROR" app.log
```

Show 2 lines of context (before and after)

```
grep -n -C 2 "ERROR" app.log
```

Find a specific request ID

```
grep "req-12345" app.log
```

Search multiple log files

```
grep -r "ERROR" /var/log/
```

5. Example Output (grep -n -C 2)

```
user@server:~$ grep -n -C 2 'ERROR' app.log
102:Oct 23 14:21:10 INFO  Starting payment process
103:Oct 23 14:21:12 INFO  Calling external API
104:Oct 23 14:21:15 ERROR Payment failed: timeout
105:Oct 23 14:21:15 WARN  Retrying request
106:Oct 23 14:21:20 INFO  Payment process ended
```

6. When to Use grep

- Find specific errors or warnings.
- Search for a request ID or user ID.
- Investigate repeated failures.
- Filter out noisy logs and focus on important events.
- Search across multiple log files or directories.

7. Practice (Hands-On)

1. View the last 50 lines of app.log.
   ```
   tail -n 50 app.log
   ```
2. Find all ERROR lines with line numbers.
   ```
   grep -n "ERROR" app.log
   ```
3. Show 2 lines before and after an error.
   ```
   grep -n -C 2 "ERROR" app.log
   ```
4. Search for a specific request ID (replace with one you find in the log).
   ```
   grep "req-12345" app.log
   ```

VERIQTA

Page 08: Reconstruct a Failure Timeline

Learn to connect log events chronologically, account for time zones, and distinguish the first observable failure from downstream errors.

Practice: Reconstruct a simulated outage from eight log entries.

1. Why Reconstruct a Timeline?

- Logs come from different components and may look confusing.
- Events happen in a sequence, and timing helps you find the real root cause.
- The first error is often the cause, while later errors are just symptoms.
- You must also consider time zones and different server clocks.

2. Key Things to Check

1. Timestamps (date, time, time zone).
2. Event order (what happened first).
3. Component or service (web server, app, database).
4. Log level (INFO, WARN, ERROR).
5. Request ID or correlation ID.
6. Related events across multiple logs.
7. First observable failure vs downstream errors.

3. Sample Log Entries (Unordered)

```
1 Oct 23 14:21:10 app[1234]: INFO Processing request id=abc123
2 Oct 23 14:21:15 db[5678]: ERROR Connection to database failed
3 Oct 23 14:21:12 app[1234]: INFO Calling external API
4 Oct 23 14:21:20 app[1234]: ERROR Payment service timeout
5 Oct 23 14:21:05 nginx[4321]: INFO 200 GET /checkout
6 Oct 23 14:21:25 app[1234]: ERROR Failed to complete order
7 Oct 23 14:21:18 api[9876]: WARN Slow response from payment service
8 Oct 23 14:21:30 worker[2468]: ERROR Job failed: retry limit reached
```

4. Sort and Build the Timeline

Time	Component	Level	Event
14:21:05	nginx	INFO	200 GET /checkout
14:21:10	app	INFO	Processing request id=abc123
14:21:12	app	INFO	Calling external API
14:21:15	db	ERROR	Connection to database failed
14:21:18	api	WARN	Slow response from payment service
14:21:25	app	ERROR	Payment service timeout
14:21:25	app	ERROR	Failed to complete order
14:21:30	worker	ERROR	Job failed: retry limit reached

5. Visual Failure Timeline



6. Identify the Root Cause

- The first observable failure is the database connection error at 14:21:15.
- Later errors (payment timeout, order failure, job failure) are downstream effects.
- Understanding the timeline prevents you from chasing the wrong problem.

7. Handle Time Zones and Clocks

- Logs may be in UTC or local time.
- Check the server time zone: `timedatectl`
- When analysing multiple systems, ensure you convert timestamps to the same time zone.
- Small clock differences between servers can affect the exact order, so look at the bigger picture.

8. Practice (Hands-On)

1. Use the eight sample log entries above.
2. Arrange them in the correct chronological order.
3. Identify the first observable failure.
4. Explain which later errors are downstream effects.

9. Useful Commands

Command	What It Does
`sort app.log`	Sort log entries by timestamp.
`grep "ERROR" app.log`	Find error messages.
`grep -n "ERROR" app.log`	Show line numbers.
`grep -n -C 2 "ERROR" app.log`	Show context lines around errors.
`date`	Check server date and time.
`timedatectl`	Check time zone and clock settings.

Page 09: Investigate Linux Services with journalctl

Learn systemd service logs, unit filters, time ranges, and severity filtering.

Practice: Inspect an available service using `journalctl -u SERVICE --since "1 hour ago"`, replacing SERVICE with its actual unit name.

1. What is journalctl?

- journalctl is used to view logs from systemd services, the kernel, and the system.
- It reads logs from the systemd journal, not from files in /var/log.
- It is useful for troubleshooting services, boot issues, and system events.
- Logs include timestamps, unit names, priorities, and detailed messages.

2. Basic Syntax

`journalctl [options] [filters]`

- `-u` : filter by a specific service (unit)
- `--since` : show logs from a specific time
- `--until` : show logs up to a specific time
- `-p` : filter by log level (priority)
- `-n` : show last N lines
- `-f` : follow logs in real time
- `--no-pager` : display output without a pager

3. Check an Available Service

```
user@server:~$ systemctl list-units --type=service \
               --state=running | head -n 10
UNIT            LOAD   ACTIVE  SUB      DESCRIPTION
ssh.service     loaded active  running  OpenSSH Server
nginx.service   loaded active  running  A high performance
mysql.service   loaded active  running  MySQL Database
docker.service  loaded active  running  Docker Application
cron.service    loaded active  running  Regular background
```

- Find a service name you want to investigate (e.g. nginx, ssh, docker, mysql).
- Use the exact unit name with `journalctl -u`.

4. View Logs for a Specific Service

```
user@server:~$ journalctl -u nginx.service
Oct 23 14:15:10 server nginx[1234]: Started nginx.
Oct 23 14:15:12 server nginx[1234]: Listening on 0.0.0.0:80
Oct 23 14:16:01 server nginx[1234]: Client connected
Oct 23 14:16:15 server nginx[1234]: GET /index.html 200
```

- Replace `nginx.service` with your actual service name.
- This shows all logs for the service.
- Use `-n` to show the last few lines.

5. Use Time Ranges

```
user@server:~$ journalctl -u nginx.service \
               --since "1 hour ago"
Oct 23 13:20:10 server nginx[1234]: Reloading config
Oct 23 13:45:22 server nginx[1234]: Client connected
Oct 23 14:10:05 server nginx[1234]: GET /api 200
Oct 23 14:15:10 server nginx[1234]: Worker process started
```

- Use `--since "1 hour ago"`, `"today"`, `"2026-10-23 10:00:00"`.
- Use `--until` to set an end time.
- This helps you focus on a specific incident period.

6. Filter by Log Level (Severity)

```
user@server:~$ journalctl -u nginx.service -p err
Oct 23 14:12:01 server nginx[1234]: ERROR open() "/etc…
Oct 23 14:15:02 server nginx[1234]: ERROR failed to load
Oct 23 14:12:05 server nginx[1234]: ERROR worker process
```

- Common log levels:
  - `emerg` : system is unusable
  - `alert` : action must be taken immediately
  - `crit` : critical conditions
  - `err` : error conditions
  - `warning` : warning messages
  - `info` : informational messages
  - `debug` : detailed debugging information

Use `-p` to filter by level, e.g. `-p err`, `-p warning`, `-p info`.

7. Follow Logs in Real Time

```
user@server:~$ journalctl -u nginx.service -f
Oct 23 14:20:01 server nginx[1234]: GET /login 200
Oct 23 14:20:05 server nginx[1234]: POST /api 200
Oct 23 14:20:10 server nginx[1234]: ERROR upstream timeout
```

- Use `-f` to watch new log entries as they happen.
- Useful during troubleshooting or while testing.
- Press Ctrl + C to stop following logs.

8. Practice (Hands-On)

1. List running services: `systemctl list-units --type=service`.
2. Choose a service (e.g. nginx, ssh, docker, mysql).
3. View the last 50 log entries:
   `journalctl -u SERVICE -n 50`
4. View logs from the last hour:
   `journalctl -u SERVICE --since "1 hour ago"`
5. Filter only error logs:
   `journalctl -u SERVICE -p err`
6. Follow the logs in real time:
   `journalctl -u SERVICE -f`

Page 10: Troubleshoot a Service That Will Not Start

Investigate failed service startup, incorrect paths, missing configuration, permissions, and port conflicts.

Practice: Diagnose a deliberately broken lab service, correct its configuration, and verify recovery.

1. Check the Service Status

```
user@server:~$ systemctl status myapp.service
● myapp.service - My Application
   Loaded: loaded (/etc/systemd/system/myapp.service; enabled)
   Active: failed (Result: exit-code) since Wed 2026-10-23 14:20:10
  Process: 1234 ExecStart=/opt/myapp/myapp (code=exited, status=203/
Main PID: 1234 (code=exited, status=203/EXEC)
```

- Shows whether the service is active, failed, or inactive.
- Look at the exact error message and exit code.
- Identify the unit file path and the command being executed.

2. View Detailed Logs with journalctl

```
user@server:~$ journalctl -u myapp.service --no-pager
Oct 23 14:20:10 server systemd[1]: Starting My Application...
Oct 23 14:20:10 server myapp[1234]: /opt/myapp/myapp: No such
                file or directory
Oct 23 14:20:10 server systemd[1]: myapp.service: Main process exited,
Oct 23 14:20:10 server systemd[1]: Failed to start My Application.
```

- Shows detailed logs for the service unit.
- Reveals the real reason for the failure.
- Look for missing files, permission errors, or configuration issues.
- Use `--since "1 hour ago"` to focus on recent attempts.

3. Common Reasons a Service Fails to Start

Cause	Example Error Message
Incorrect path	/opt/myapp/myapp: No such file or directory
Missing config file	Failed to load configuration file
Permission denied	Permission denied
Port already in use	Address already in use (port 8080)
Invalid unit file	Failed to parse unit file
Missing dependency	Dependency failed for some.service
Wrong user/group	Failed at step USER spawning /opt/myapp
Environment variable	Environment variable not set

4. Check the Service Configuration

```
user@server:~$ sudo cat /etc/systemd/system/myapp.service
[Unit]
Description=My Application
After=network.target

[Service]
User=appuser
ExecStart=/opt/myapp/myapp
WorkingDirectory=/opt/myapp
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

- Verify that ExecStart path exists and is correct.
- Check WorkingDirectory, User, and environment variables.
- Look for typos, missing files, or incorrect permissions.

5. Inspect Files and Permissions

```
user@server:~$ ls -l /opt/myapp/
-rwxr-xr-x 1 appuser appuser 123456 Oct 23 14:10 myapp
user@server:~$ ls -l /etc/myapp/
-rw-r----- 1 appuser appuser 2048 Oct 23 14:10 config.yaml
```

- Ensure the application binary and config files exist.
- Check file permissions and ownership.
- Make sure the service user can read the required files.

6. Check for Port Conflicts

```
user@server:~$ sudo ss -tulnp | grep :8080
LISTEN    0      128          0.0.0.0:8080       0.0.0.0:*
users:(("nginx",pid=987,fd=6))
```

- Check if the required port is already in use.
- Stop or reconfigure the conflicting service.
- Use a different port if needed.

7. Fix the Issue and Reload the Service

1. Edit the configuration or unit file:
   `sudo nano /etc/systemd/system/myapp.service`
2. Reload systemd:
   `sudo systemctl daemon-reload`
3. Start the service:
   `sudo systemctl start myapp.service`
4. Check the status:
   `sudo systemctl status myapp.service`

8. Verify Recovery

```
user@server:~$ systemctl status myapp.service
● myapp.service - My Application
   Loaded: loaded (/etc/systemd/system/myapp.service; enabled)
   Active: active (running) since Wed 2026-10-23 14:35:10
 Main PID: 2345 (myapp)
    Tasks: 5 (limit: 2314)
   Memory: 50.2M
```

- Service should show active (running).
- Check recent logs to confirm normal operation.
- Test the application to ensure it is working as expected.

Page 11: Investigate HTTP Errors with NGINX Logs

Understand access logs, error logs, HTTP status codes, request paths, and response times.
Practice: Generate successful and failed HTTP requests using curl, then locate their entries in NGINX logs.

1. NGINX Log Files

Log File	What It Contains
/var/log/nginx/access.log	Information about every HTTP request (client IP, method, path, status code, response time, user agent).
/var/log/nginx/error.log	Error messages and warnings from NGINX (startup issues, configuration errors, upstream errors, etc.).

2. NGINX Access Log Format (Common)

```
192.168.1.10 - - [23/Oct/2026:14:21:10 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/8.0.1" 0.003
```

1. Client IP address
2. Timestamp (date and time)
3. HTTP method and request path
4. HTTP status code
5. Response size (bytes)
6. User agent (client)
7. Response time (seconds)

3. Common HTTP Status Codes

Code	Meaning	Examples
200	OK (Success)	Request completed successfully
301	Moved Permanently	URL redirected
302	Found (Redirect)	Temporary redirect
400	Bad Request	Invalid request
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Requested resource not found
500	Internal Server Error	Server-side error
502	Bad Gateway	Upstream server error
503	Service Unavailable	Service temporarily down

4. Generate Test Requests with curl

```
user@server:~$ # Successful request (200)
user@server:~$ curl http://localhost/
user@server:~$ # Not found (404)
user@server:~$ curl http://localhost/notfound
user@server:~$ # Trigger server error (500) - example
user@server:~$ curl http://localhost/error
```

- Use different paths to generate different status codes.
- These requests will be written to the NGINX access log.
- If there is an error, details may also appear in the error log.

5. View Access Logs

```
user@server:~$ tail -n 10 /var/log/nginx/access.log
192.168.1.10 - - [23/Oct/2026:14:20:15 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/8.0.1" 0.003
192.168.1.10 - - [23/Oct/2026:14:21:02 +0000] "GET /notfound HTTP/1.1" 404 153 "-" "curl/8.0.1" 0.001
192.168.1.10 - - [23/Oct/2026:14:21:30 +0000] "GET /error HTTP/1.1" 500 612 "-" "curl/8.0.1" 0.004
```

- Look for the timestamp, request path, status code and response time.
- You can use less /var/log/nginx/access.log to scroll through logs.

6. View Error Logs

```
user@server:~$ tail -n 10 /var/log/nginx/error.log
2026/10/23 14:21:30 [error] 12345#12345: *5 open() "/usr/share/nginx/html/error" failed (2: No such file or directory), client: 192.168.1.10, server: _, request: "GET /error HTTP/1.1", host: "localhost"
```

- Shows detailed error messages.
- Helps you understand why a request failed.
- Useful for debugging configuration and upstream issues.

7. Useful Commands

Command	What It Does
tail -f /var/log/nginx/access.log	Follow access logs in real time
grep " 404 " /var/log/nginx/access.log	Find 404 errors
grep " 500 " /var/log/nginx/access.log	Find 500 errors
less /var/log/nginx/access.log	View logs page by page
tail -f /var/log/nginx/error.log	Follow error logs in real time
grep -i error /var/log/nginx/error.log	Find error messages (case-insensitive)

8. Practice (Hands-On)

1. Generate a successful request:
   `curl http://localhost/`
2. Generate a failed request (404):
   `curl http://localhost/notfound`
3. Locate the requests in the access log:
   `tail -n 20 /var/log/nginx/access.log`
4. If there are errors, check the error log:
   `tail -n 20 /var/log/nginx/error.log`

Page 12: Find the Cause of a 502 Bad Gateway

Investigate a reverse proxy that cannot successfully communicate with its upstream application.

Practice: Stop a disposable backend, reproduce the error, compare proxy and application evidence, then restore and verify service.

1. What is a 502 Bad Gateway?

- A 502 Bad Gateway means the reverse proxy (NGINX) received an invalid response from the upstream server.
- It usually happens when the upstream application is down, unreachable, or not responding correctly.
- The client (browser) connects to NGINX, but NGINX cannot communicate with the backend.

2. Architecture Overview



3. Reproduce the 502 Error

```
user@server:~$ # Check current status
user@server:~$ systemctl status myapp
● myapp.service - My Application
     Active: active (running)
user@server:~$ # Stop the backend application
user@server:~$ sudo systemctl stop myapp
user@server:~$ # Send a request through NGINX
user@server:~$ curl -i http://localhost/
HTTP/1.1 502 Bad Gateway
Server: nginx/1.24.0
Date: Thu, 23 Oct 2026 14:22:10 GMT
Content-Type: text/html
Content-Length: 157
```

- Stop the backend application (upstream).
- Send a request to NGINX.
- You should get a 502 Bad Gateway response.

4. Check NGINX Error Logs

```
user@server:~$ sudo tail -n 10 /var/log/nginx/error.log
2026/10/23 14:22:10 [error] 12345#12345: *1 connect() failed (111: Connection refused) while connecting to upstream, client: 192.168.1.10, server: _, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:3000/", host: "localhost"
```

- Look for connect() failed or upstream errors.
- This shows NGINX cannot reach the backend.
- Check the upstream address and port.

5. Check Application Logs

```
user@server:~$ sudo journalctl -u myapp --since "1 hour ago" --no-pager
● myapp.service - My Application
     Active: inactive (dead) since Thu 2026-10-23 14:20:10
   Process: 1234 ExecStart=/usr/bin/node /opt/myapp/app.js
  Main PID: 1234 (code=exited, status=1/FAILURE)
Oct 23 14:20:10 server systemd[1]: myapp.service: Stopped with result 'exit-code'.
```

- Confirm the application is stopped or crashed.
- Look for startup errors, missing dependencies, or port issues.
- Compare the time with the NGINX error log.

6. Common Causes of 502 Errors

Cause	Example
Upstream application down	Service crashed or stopped
Wrong upstream port	NGINX pointing to wrong port
Application slow or hanging	Not responding in time
Firewall blocking connection	Port not accessible
Incorrect upstream address	Wrong IP or hostname
Resource issues	High CPU, memory, or file limits
Application errors	App fails to start properly

7. Fix the Issue and Restore Service

```
user@server:~$ # Start the backend application
user@server:~$ sudo systemctl start myapp
user@server:~$ # Verify it is running
user@server:~$ systemctl status myapp | head -n 5
● myapp.service - My Application
     Active: active (running) since Thu 2026-10-23 14:30:15
```

- Start the backend service.
- Confirm it is running and listening on the correct port.
- Test again with curl and ensure you get a 200 OK response.

8. Verify Recovery

```
user@server:~$ curl -i http://localhost/
HTTP/1.1 200 OK
Server: nginx/1.24.0
Date: Thu, 23 Oct 2026 14:31:20 GMT
Content-Type: text/html
user@server:~$ tail -n 5 /var/log/nginx/access.log
192.168.1.10 - - [23/Oct/2026:14:31:20 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/8.0.1" "0.002"
```

- Send a request again and confirm 200 OK.
- Check access logs to see the successful request.
- Make sure both NGINX and the application are healthy.

Page 13: Investigate Docker Container Logs

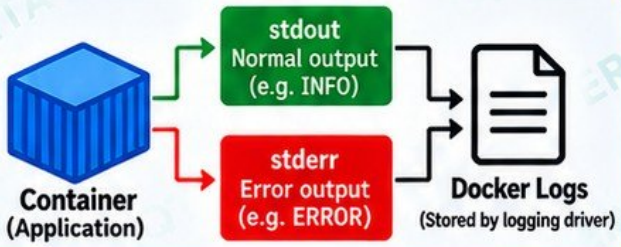
Understand standard output, standard error, container names, timestamps, and Docker logging drivers.

Practice: Run a containerized lab application and inspect it using `docker logs --tail 50 --timestamps CONTAINER`.

1. What are Docker Container Logs?

- Docker captures the standard output (stdout) and standard error (stderr) from containers.
- Logs help you understand application behaviour, errors, and startup issues.
- You can view logs using the container name or ID.
- Docker supports different logging drivers (json-file, syslog, journald, etc.).

2. How Docker Logging Works



3. List Running Containers

```
user@server:~$ docker ps
CONTAINER ID   IMAGE          COMMAND          STATUS          NAMES
1a2b3c4d5e6f   nginx:latest   "nginx -g ..."   Up 5 minutes    web
7f8e9d0c1b2a   myapp:latest   "node app.js"    Up 5 minutes    app
```

- Shows running containers and their names.
- You can use either the container name or ID with docker logs.

4. View Container Logs (Basic)

```
user@server:~$ docker logs app
Server started on port 3000
Connected to database
Listening for requests...
GET / 200 12ms
GET /api/users 200 8ms
ERROR Failed to fetch user data: Connection timeout
```

- Displays both stdout and stderr from the container.
- Shows recent log entries.

5. View Logs with Timestamps and Tail

```
user@server:~$ docker logs --tail 50 --timestamps app
2026-10-23T14:20:01.123456789Z  Server started on port 3000
2026-10-23T14:20:05.234567890Z  GET / 200 12ms
2026-10-23T14:21:10.345678901Z  GET /api/users 200 8ms
2026-10-23T14:22:15.456789012Z  ERROR Failed to fetch user data:
Connection timeout
```

- `--tail 50` shows the last 50 log lines.
- `--timestamps` adds the exact time for each log entry.
- Useful for truubleshooting and correlating events.

6. Follow Logs in Real Time

```
user@server:~$ docker logs -f app
2026-10-23T14:23:10.111111111Z  GET / 200 10ms
2026-10-23T14:23:15.222222222Z  GET /api/users 200 9ms
2026-10-23T14:23:20.333333333Z  ERROR Database connection
                                failed: timeout
2026-10-23T14:23:25.444444444Z  Retrying connection...
```

- `--f` follows new log entries as they happen.
- Useful while testing or reproducing issues.
- Press `Ctrl + C` to stop following logs.

7. View Only Errors, or Use Filters

```
# View only error messages
user@server:~$ docker logs app 2>&1 | grep -i error
2026-10-23T14:22:15.456789012Z  ERROR Failed to fetch user data
2026-10-23T14:23:20.333333333Z  ERROR Database connection failed
# Show last 20 lines
user@server:~$ docker logs --tail 20 app
# Show logs since a specific time
user@server:~$ docker logs --since "1 hour ago" app
```

- You can use grep to find specific errors or patterns.
- You can view logs from a specific time range.

8. Check the Logging Driver

```
user@server:~$ docker inspect app --format '{{.HostConfig.\
               LogConfig.Type}}'
json-file
```

- Shows which logging driver is being used.
- Common drivers: json-file (default), syslog, journald, fluentd, splunk, awslogs, etc.

9. Practice (Hands-On)

1. Run a containerized lab application (e.g. nginx or a sample app).
2. View its logs using: `docker logs --tail 50 --timestamps CONTAINER`.
3. Generate some requests (successful and failed).
4. Use filters (grep) to find errors.
5. Follow the logs in real time while generating more requests.

Page 14: Troubleshoot a Container That Keeps Exiting

Investigate application exceptions, missing environment variables, bad startup commands, and exit codes.

Practice: Introduce a faulty configuration into the lab container, examine **docker logs** and **docker inspect**, then correct and verify it.

1. What Does "Container Keeps Exiting" Mean?

- The container starts but quickly stops and restarts (repeatedly exits).
- This is usually caused by application errors, missing environment variables, bad commands, or configuration issues.
- Check logs, exit codes, and container configuration to find the root cause.
- Use **docker logs**, **docker inspect**, and **docker ps** to investigate.

2. Example: Container Keeps Exiting



Common Causes

- Application exception
- Missing environment variables
- Bad startup command
- Incorrect file path
- Permission issues
- Configuration errors
- Service dependency not available
- Non-zero exit code

3. Check Container Status

```
user@server:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND          STATUS
f8a2c1d9e123   myapp:latest   "node app.js"    Exited (1) 5 seconds ago
```

- **Exited** indicates the container stopped.
- (1) is the exit code (non-zero means failure).
- Use **docker ps -a** to see the exit code and restart count.

4. View Container Logs

```
user@server:~$ docker logs myapp
Error: Missing required environment variable: DB_HOST
  at Object.<anonymous> (/app/index.js:10:9)
  at Module._compile (node:internal/modules/cjs/loader:1254:14)
Node.js v20.10.0
Container exiting with code 1
```

- Logs show the application error or exception.
- Look for stack traces, missing variables, or file not found errors.
- Use **docker logs --tail 50 myapp** for recent logs.

5. Inspect the Container for More Details

```
user@server:~$ docker inspect myapp | grep -E
"ExitCode|RestartCount|State" -A 5
    "State": {
        "Status": "exited",
        "ExitCode": 1,
        "Error": "",
        "StartedAt": "2026-10-23T14:55:10.123456789Z",
        "FinishedAt": "2026-10-23T14:55:12.456789123Z"
    },
    "RestartCount": 5,
```

- Check **ExitCode** to see why it stopped.
- **RestartCount** shows how many times it has restarted.
- Use **docker inspect** to view environment variables, command, volumes, and configuration.

6. Common Causes and How to Identify

Cause	How to Identify	Example / Evidence
Missing environment variable	Check logs and inspect env section	Error: Missing DB_HOST
Bad startup command	Check docker inspect or Dockerfile	exec: "npm": executable file not found
Application exception	Check container logs	Stack trace in logs
Incorrect file path	Check logs	No such file or directory
Permission issues	Check logs	EACCES: permission denied
Service dependency not available	Check logs and network	Connection refused
Non-zero exit code	Check docker ps -a or inspect	Exited (1), (137), etc.

7. Fix the Issue and Verify

1. Identify the root cause from logs and inspect output.
2. Fix the configuration (e.g. set the missing environment variable).
3. Start the container again:
   ```
   docker run -d --name myapp -e DB_HOST=localhost myapp:latest
   ```
4. Check that it is running:
   ```
   docker ps | grep myapp
   ```
5. View the logs to confirm it is working:
   ```
   docker logs --tail 20 myapp
   ```

8. Practice (Hands-On)

1. Run a containerized lab application with a deliberate error (e.g. missing environment variable):
   ```
   docker run -d --name myapp myapp:latest
   ```
2. Check the container status and exit code:
   ```
   docker ps -a
   ```
3. Examine the logs:
   ```
   docker logs myapp
   ```
4. Inspect the container details:
   ```
   docker inspect myapp
   ```
5. Fix the configuration, start it again, and verify it is running.

Page 15: Read Kubernetes Pod Logs

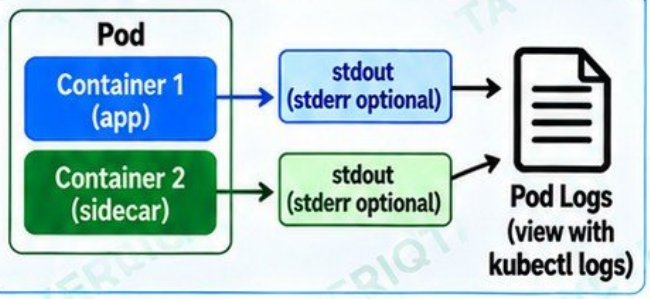
Understand Pod logs, container selection, namespaces, timestamps, and why application logs may disappear when containers are replaced.

Practice: Deploy a small application to a local Kubernetes cluster and inspect its logs using kubectl logs.

1. What are Kubernetes Pod Logs?

- Pod logs show the standard output (stdout) and standard error (stderr) from containers.
- Each container in a Pod has its own logs.
- Logs help you troubleshoot application issues, see startup messages, and understand failures.
- Logs are stored on the node. When a container is replaced or deleted, the previous logs may disappear unless a logging solution is used.

2. Kubernetes Architecture (Logs Flow)



3. Deploy a Sample Application

```
user@server:~$ kubectl create deployment webapp \
  --image=nginx:latest --replicas=1
user@server:~$ kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
webapp-7d6f9c8b7-abc12   1/1     Running   0          30s
```

- Deploys a simple NGINX application in your cluster.
- Check that the Pod is running before viewing logs.

4. View Pod Logs (Basic)

```
user@server:~$ kubectl logs webapp-7d6f9c8b7-abc12
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty,
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-
/docker-entrypoint.sh: Configuration complete; ready for
2026/10/23 14:12:10 [notice] 1#1: using the "epoll" event
2026/10/23 14:12:10 [notice] 1#1: nginx/1.25.3
2026/10/23 14:12:10 [notice] 1#1: start worker processes
2026/10/23 14:12:10 [notice] 1#1: start worker process 29
```

- Displays logs from the default container.
- Shows application startup messages and runtime logs.
- Use the full Pod name or a shorter name (kubectl logs webapp).

5. Common kubectl logs Options

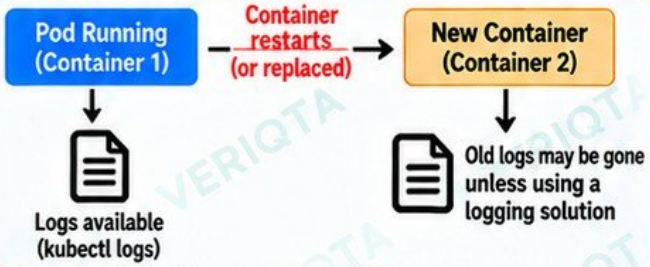
Command	What It Does
kubectl logs POD_NAME	View logs from a Pod
kubectl logs POD -c CONTAINER	View logs from a specific container
kubectl logs -n NAMESPACE POD	View logs in a specific namespace
kubectl logs --tail 50 POD	Show last 50 log lines
kubectl logs -f POD	Follow logs in real time
kubectl logs --timestamps POD	Show timestamps with logs
kubectl logs --previous POD	View logs from the previous container (instance that crashed)

6. View Logs from a Specific Container

```
user@server:~$ kubectl logs webapp-7d6f9c8b7-abc12 \
  -c nginx --tail 20 --timestamps
2026-10-23T14:15:10.123456789Z 172.17.0.1 - - [23/Oct/
2026-10-23T14:15:12.987654321Z 172.17.0.1 - - [23/Oct/
2026-10-23T14:15:15.111111111Z 172.17.0.1 - - [23/Oct/
```

- Use -c to select a container when a Pod has multiple containers.
- Use --timestamps to see the exact time of each log entry.
- Use --tail to view only the most recent lines.

7. Why Logs May Disappear



- Logs are stored on the node for the container.
- When a container is replaced, previous logs are lost.
- Use a centralized logging solution (e.g. Fluentd, Loki, ELK) for long-term logs.

8. Practice (Hands-On)

1. Deploy a small application:
   `kubectl create deployment webapp --image=nginx:latest --replicas=1`
2. Check the Pod and view its logs:
   `kubectl logs webapp`
3. Follow the logs in real time:
   `kubectl logs -f webapp`
4. Generate some requests:
   `kubectl exec -it webapp -- curl -s http://localhost > /dev/null`
5. Restart the Pod and check previous logs:
   `kubectl rollout restart deployment/webapp`
   `kubectl logs --previous -l app=webapp`

Page 16: Investigate Kubernetes CrashLoopBackOff

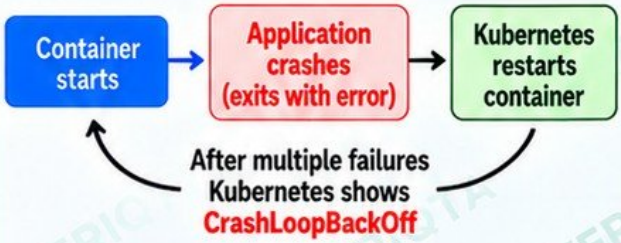
Learn why containers restart, how to inspect previous container logs, and when Kubernetes events provide additional evidence.

Practice: Deploy an intentionally failing application, inspect `kubectl describe pod`, retrieve previous logs using `kubectl logs POD --previous`, and verify the correction.

1. What is CrashLoopBackOff?

- CrashLoopBackOff means a container is starting but repeatedly crashing.
- Kubernetes restarts the container automatically with an increasing backoff delay.
- Common causes: application errors, missing environment variables, bad commands, missing files, permission issues, or dependency failures.
- Use pod status, logs, and events to find the root cause.

2. How It Works



3. See CrashLoopBackOff in Pod Status

```
user@server:~$ kubectl get pods
NAME                     READY   STATUS             RESTARTS   AGE
badapp-7d9f8c6b9-abc12   0/1     CrashLoopBackOff   5          2m
```

- **STATUS** shows **CrashLoopBackOff** when the container keeps crashing.
- **RESTARTS** increases each time the container restarts.

4. Describe the Pod for Events and Details

```
user@server:~$ kubectl describe pod badapp-7d9f8c6b9-abc12
Events:
Type     Reason     Age   From               Message
Normal   Scheduled  2m    default-scheduler  Successfully assigned pod to node
Normal   Pulling    2m    kubelet            Pulling image "badapp:latest"
Normal   Pulled     1m    kubelet            Successfully pulled image
Normal   Created    1m    kubelet            Created container: badapp
Warning  BackOff    30s   kubelet            Back-off restarting failed container badapp
```

- Look for Warning events (BackOff, Failed, Error).
- Events show why the container is restarting.
- Check the exact error message and reason.

5. View Current and Previous Logs

```
# Current logs (may be empty if container crashes quickly)
user@server:~$ kubectl logs badapp-7d9f8c6b9-abc12
Error: cannot find config file: /app/config.yaml

# Previous logs (from the last crashed container)
user@server:~$ kubectl logs badapp-7d9f8c6b9-abc12 --previous
Error: cannot find config file: /app/config.yaml
```

- Use `--previous` to see logs from the last crashed container.
- Logs often show the real application error.

6. Common Causes and Evidence

Cause	Evidence (Logs/Events)	Example
Missing environment variable	Error in logs	env: DB_HOST not set
Bad startup command	Error in logs	exec: "npm": executable file not found
Missing configuration file	Error in logs	cannot find /app/config.yaml
Permission issues	Error in logs	permission denied
Application exception	Stack trace in logs	Traceback... (crash)
Dependency not available	Connection errors	could not connect to db:5432

7. Fix the Issue and Verify

1. Check and fix the configuration (e.g. add missing environment variable).
2. Re-deploy or restart the Pod:
   ```
   kubectl rollout restart deployment badapp
   ```
3. Check the Pod status:
   ```
   kubectl get pods
   ```
4. Verify it is running:
   ```
   kubectl logs -f badapp-7d9f8c6b9-abc12
   ```
5. Confirm no more errors in the logs.

8. Practice (Hands-On)

1. Deploy a failing application (missing environment variable):
   ```
   kubectl create deployment badapp --image=nginx \
   --env=DB_HOST=missing --replicas=1
   ```
2. Check the Pod and identify CrashLoopBackOff:
   ```
   kubectl get pods
   ```
3. Describe the Pod and review events:
   ```
   kubectl describe pod POD_NAME
   ```
4. View previous logs:
   ```
   kubectl logs POD_NAME --previous
   ```
5. Fix the issue (set correct environment variable) and verify:
   ```
   kubectl set env deployment/badapp DB_HOST=localhost
   kubectl rollout restart deployment/badapp
   ```

Page 17: Follow One Request Across Multiple Services

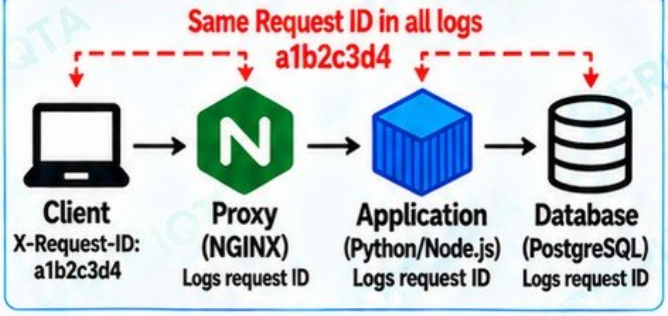
Understand correlation IDs, request IDs, structured JSON logs, and why a single error may appear in several services.

Practice: Follow a simulated request through a proxy, application, and database using a shared request ID.

1. What Are Correlation IDs?

- A correlation ID (also called a request ID) is a unique identifier for a single request as it travels through multiple services.
- It helps you connect log entries from different services.
- It is usually passed in a header (e.g. X-Request-ID) and included in logs.
- Logs are often in structured JSON format, making it easier to search by request ID.

2. Request Flow Across Multiple Services



3. Generate a Request with a Correlation ID

```
user@server:~$ curl -H "X-Request-ID: a1b2c3d4" \
  http://localhost/api/users
{
  "status": "error",
  "message": "Database connection failed",
  "request_id": "a1b2c3d4"
}
```

- Send a request with a custom X-Request-ID header.
- The same request ID should appear in logs for all services.
- Note the request ID for log investigation.

4. Check Proxy Logs (NGINX)

```
user@server:~$ sudo grep "a1b2c3d4" /var/log/nginx/access.log
192.168.1.10 - - [23/Oct/2026:14:50:10 +0000] "GET /api/users
HTTP/1.1" 500 172 "-" "curl/8.0.1" "a1b2c3d4"
```

- Find the request ID in the NGINX access log.
- Confirm the request reached the application.
- Note the timestamp, status code, and response time.

5. Check Application Logs (JSON Format)

```
user@server:~$ docker logs app | grep a1b2c3d4
{"time":"2026-10-23T14:50:10Z","level":"INFO",
 "request_id":"a1b2c3d4","message":"Received request",
 "path":"/api/users"}
{"time":"2026-10-23T14:50:10Z","level":"ERROR",
 "request_id":"a1b2c3d4","message":"Database connection
 failed","error":"ECONNREFUSED"}
```

- Search for the request ID in application logs.
- Look for errors, exceptions, or stack traces.
- Structured JSON logs make it easy to find related events.

6. Check Database Logs (PostgreSQL)

```
user@server:~$ sudo grep a1b2c3d4 /var/log/postgresql/postgresql.log
2026-10-23 14:50:10.123 UTC [5432] [a1b2c3d4] ERROR:
  connection to client 192.168.1.20 failed: Connection refused
2026-10-23 14:50:10.124 UTC [5432] [a1b2c3d4] DETAIL:
  could not accept SSL connection
```

- Find the request ID in database logs.
- Confirm the error and exact timestamp.
- This shows the database was the root cause.

7. Example: Same Request ID Across All Services

Service	Log Entry (Simplified)	What It Shows
NGINX (Proxy)	a1b2c3d4 GET /api/users 500	Request reached app
Application	a1b2c3d4 Database connection failed	App tried to connect to database
PostgreSQL	a1b2c3d4 Connection refused	Database was unreachable

8. Why a Single Error Appears in Multiple Services

- A single request passes through multiple services.
- Each service logs its own part of the request.
- The same error (e.g. database failure) appears in proxy, application, and database logs with the same request ID.
- Correlating logs helps you see the full picture and find the real root cause quickly.

9. Practice (Hands-On)

1. Deploy a simple application (e.g. NGINX → app → PostgreSQL).
2. Send a request with a custom X-Request-ID header.
3. Find the request ID in proxy, application, and database logs.
4. Identify the root cause from the logs.
5. Fix the issue and verify the request now succeeds.

Page 18: Detect Patterns, Spikes, and Repeated Failures

Use basic Linux tools to count errors, group repeated messages, identify failing endpoints, and compare logs before and after a change.

Practice: Analyze a supplied log dataset using grep, sort, uniq -c, and awk, with explanations of the assumptions behind each command.

1. What We Are Looking For

- Find and count error messages.
- Group repeated log entries.
- Identify the most failing endpoints.
- Detect spikes in errors over time.
- Compare logs before and after a change.
- Use grep, sort, uniq -c, and awk to analyze logs.

2. Sample Log Entries (access.log)

```
192.168.1.10 - - [23/Oct/2026:14:01:12] "GET /api/users 200"
192.168.1.11 - - [23/Oct/2026:14:01:15] "GET /api/login 401"
192.168.1.12 - - [23/Oct/2026:14:01:16] "POST /api/users 500"
192.168.1.10 - - [23/Oct/2026:14:02:01] "GET /api/users 200"
192.168.1.11 - - [23/Oct/2026:14:02:05] "GET /api/login 401"
192.168.1.12 - - [23/Oct/2026:14:02:08] "POST /api/users 500"
```

- Each line contains an IP, timestamp, request, endpoint and status code.
- We will use this sample to find patterns and failures.

3. Count Total Errors

```
user@server:~$ grep " 500 " access.log | wc -l
42
```

- Counts all lines with HTTP 500 status code.
- You can change 500 to 404, 401, etc.

4. Find and Group Repeated Messages

```
user@server:~$ grep "error" app.log | sort | uniq -c | sort -nr
  25   ERROR Database connection failed
  18   ERROR Invalid token
  10   ERROR Timeout while connecting to service
   5   ERROR User not found
```

- grep finds lines with the keyword "error".
- sort groups identical messages.
- uniq -c counts each unique message.
- sort -nr shows the highest count first.

5. Identify the Most Failing Endpoints

```
user@server:~$ awk '{print $7, $9}' access.log | \
grep -E " 4[0-9]{2}| 5[0-9]{2}" | sort | uniq -c | sort -nr
 35  /api/users 500
 28  /api/login 401
 12  /api/orders 503
  8  /api/products 404
```

- Extracts the endpoint (field 7) and status code (field 9).
- Filters only 4xx and 5xx errors.
- Groups and counts the failing endpoints.
- Helps you see which endpoint has the most failures.

6. Detect Spikes in Errors Over Time

```
user@server:~$ awk '{print $4}' access.log | \
cut -d: -f1,2 | sort | uniq -c | sort -nr | head
 120  23/Oct/2026:14
  85  23/Oct/2026:13
  40  23/Oct/2026:12
  12  23/Oct/2026:11
```

- Extracts the date and hour from the timestamp.
- Groups by hour and counts errors.
- Helps identify when errors spiked.

7. Compare Logs Before and After a Change

Before the change

```
user@server:~$ grep " 500 " \
access_before.log | wc -l
95
```

After the change

```
user@server:~$ grep " 500 " \
access_after.log | wc -l
12
```

- Compare the number of errors before and after a deployment.
- A lower count indicates an improvement.
- You can also compare specific endpoints or error messages.

8. Practice (Hands-On)

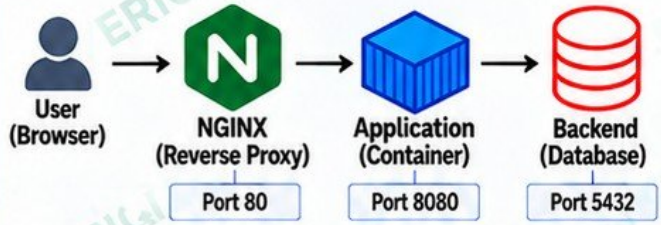
1. Download or create the sample log file (access.log and app.log).
2. Count the total number of 500 errors.
3. Find the top 5 most common error messages.
4. Identify the top 5 failing endpoints (4xx and 5xx).
5. Detect the hour with the highest number of errors.
6. Compare the error count before and after a simulated change.
7. Explain what each command does and what assumptions it makes (e.g. log format, field position, status code pattern).

Page 19: Production Incident Investigation Lab

Investigate a realistic simulated application outage involving NGINX, an application container, and a failing backend dependency.

Practice: Establish a timeline, collect logs, identify the primary failure, implement a safe correction, and verify the application from the user's perspective.

1. Lab Environment Overview



- A user accesses the application through NGINX.
- NGINX forwards requests to the application container.
- The application depends on a backend database.
- We will simulate a failure in the backend and investigate the resulting application outage.

2. The Incident Scenario

Incident: Users report that the application is not working. The website shows a 502 Bad Gateway error.

Your Tasks:

1. Confirm the issue from the user's perspective.
2. Establish a timeline of events.
3. Collect and analyze logs from NGINX, the application container, and the database.
4. Identify the primary failure.
5. Implement a safe correction.
6. Verify the application is working again.

3. Confirm the Issue (User's Perspective)

```
user@server:~$ curl -I http://localhost
HTTP/1.1 502 Bad Gateway
Server: nginx/1.24.0
Date: Thu, 23 Oct 2026 14:32:10 GMT
Content-Type: text/html
Content-Length: 157
```

- HTTP 502 confirms the application is not reachable through NGINX.
- Next, check NGINX, application, and database logs.

4. Check NGINX Logs

```
user@server:~$ sudo tail -n 10 /var/log/nginx/error.log
2026/10/23 14:32:10 [error] 1024#1024: *15 connect() failed (111: Connection refused) while connecting to upstream, client: 192.168.1.10, server: _, request: "GET / HTTP/1.1", upstream: "http://app:8080/", host: "localhost"
```

- NGINX cannot connect to the application container.
- This indicates the application is unavailable or crashing.

5. Check Application Container Logs

```
user@server:~$ docker logs app --tail 20
2026-10-23 14:32:09 [INFO] Starting application...
2026-10-23 14:32:09 [ERROR] Database connection failed: connect ECONNREFUSED 172.17.0.3:5432
2026-10-23 14:32:09 [ERROR] Exiting application...
```

- Application cannot connect to the database.
- This explains why NGINX returns 502.

6. Check Database Status

```
user@server:~$ docker ps --filter name=db
CONTAINER ID   IMAGE          COMMAND            STATUS          NAMES
a1b2c3d4e5f6   postgres:15    "docker-entryp.."  Exited (1) db
user@server:~$ docker logs db --tail 10
FATAL:  database files are incompatible with server
LOG:  database system is shut down
```

- The database container has crashed and is not running.
- This is the primary failure causing the outage.

7. Fix the Issue

```
user@server:~$ docker start db
db
user@server:~$ docker ps --filter name=db
CONTAINER ID   IMAGE          COMMAND            STATUS          NAMES
a1b2c3d4e5f6   postgres:15    "docker-entryp.."  Up 10 seconds   db
```

- Start the database container (or fix the configuration).
- Ensure the application container reconnects to the database.

8. Verify Recovery

```
user@server:~$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.24.0
Date: Thu, 23 Oct 2026 14:45:18 GMT
Content-Type: text/html
Content-Length: 612
```

- HTTP 200 confirms the application is working again.
- Users can now access the application successfully.

Page 20: Final Log Detective Challenge

Complete an independent troubleshooting assessment using an unfamiliar log dataset with multiple misleading symptoms.

Practice: Produce an incident report containing the failure timeline, supporting evidence, confirmed root cause, corrective action, recovery verification, and prevention recommendations.

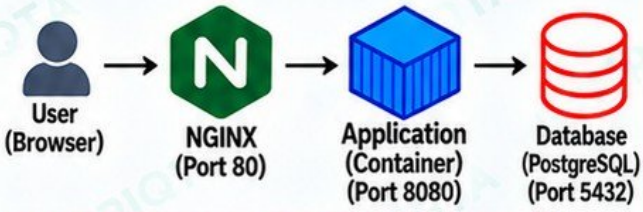
1. Challenge Scenario

You are given an unfamiliar log dataset from a real-world application environment that includes:

- NGINX (reverse proxy) logs
- Application container logs
- Database logs
- Kubernetes events
- Multiple misleading symptoms

Goal: Investigate the issue, find the real root cause, fix it, and write a complete incident report.

2. Environment Architecture



Possible Symptoms (not all are the root cause):

- Users see 502/504 errors
- Application logs show connection errors
- Database logs show timeouts
- Container restarts
- Some logs contain misleading errors

3. Investigate the Logs

Check NGINX logs:

```
user@server:~$ sudo tail -n 50 /var/log/nginx/access.log
user@server:~$ sudo tail -n 50 /var/log/nginx/error.log
```

Check application container logs:

```
user@server:~$ docker logs app --tail 50 --timestamps
```

Check database logs:

```
user@server:~$ docker logs db --tail 50
```

Check Kubernetes events (if using K8s):

```
user@server:~$ kubectl get pods
user@server:~$ kubectl describe pod <pod-name>
```

- Look for timestamps, error messages, and patterns.
- Compare logs across services.
- Identify which errors are related and which are misleading.

4. Analyze and Identify the Root Cause

1. Establish a timeline of events from the logs.
2. Identify the first failure and how it propagated.
3. Use Linux tools to analyze patterns:

```
grep "error" app.log | sort | uniq -c | sort -nr
grep "502" /var/log/nginx/access.log | wc -l
awk '{print $7}' access.log | sort | uniq -c | sort -nr
```

4. Determine the real root cause (ignore misleading errors).
5. Explain why other errors appeared.

5. Implement the Fix

```
# Example: restart a failed service
user@server:~$ docker restart app
# Example: fix a misconfiguration
user@server:~$ nano /etc/nginx/nginx.conf
user@server:~$ docker compose up -d
```

- Apply a safe and targeted fix.
- Avoid making unnecessary changes.
- Ensure all services are healthy after the fix.

6. Verify Recovery

```
user@server:~$ curl -I http://localhost
HTTP/1.1  200 OK
user@server:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
app-6f5d7c9c  1/1     Running   0          5m
db-7c8f9d6b   1/1     Running   0          5m
```

- Confirm the application is working for the user.
- Check that there are no new errors in the logs.
- Verify all services are running and stable.

7. Incident Report (Required Output)



- Incident summary
- Timeline of events (with timestamps)
- Impact on users
- Evidence from logs (screenshots/commands)
- Confirmed root cause
- Corrective action taken
- Recovery verification (e.g. curl output)
- Prevention recommendations